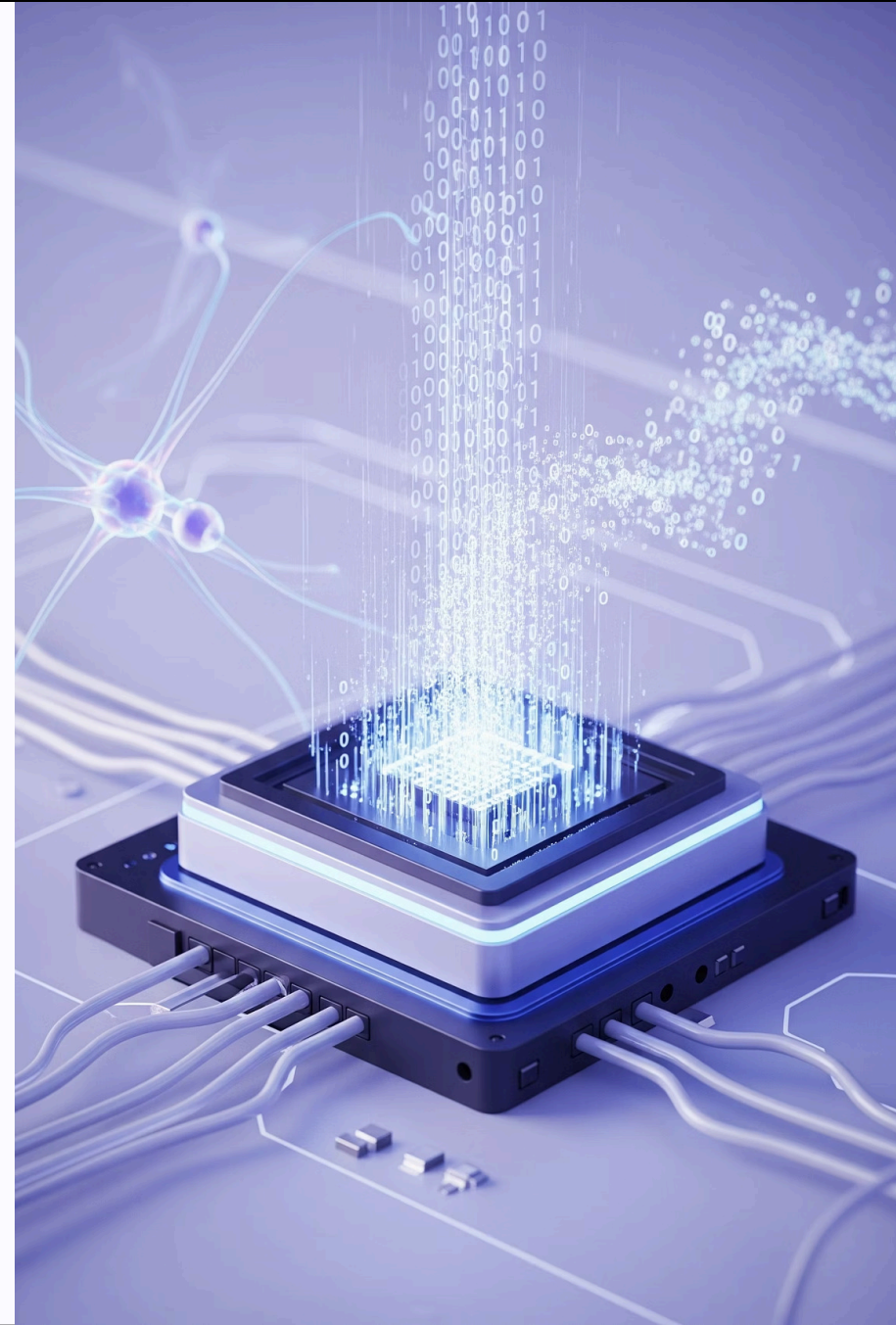


Mixed Precision Training with AMP

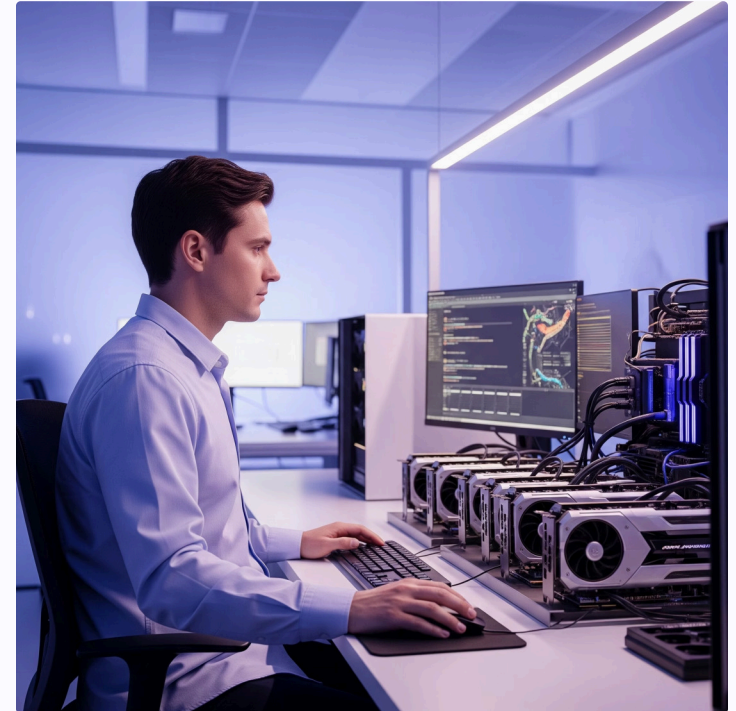
Unlock faster training with less memory by balancing FP16 and FP32 computations - a key technique for modern large-scale deep learning.



Why Mixed Precision?

Deep learning models increasingly demand enormous computational resources and memory:

- Standard FP32 (32-bit floating point) results in slower training and larger memory footprint
- Modern GPUs (especially NVIDIA Tensor Cores) are specifically optimized for FP16 operations
- The ultimate goal: **Train faster without sacrificing accuracy**



What is Mixed Precision?

1

FP16 (Half Precision)

16-bit floating point format enables:

- Faster matrix multiplications
- Reduced memory requirements
- Better utilization of specialized hardware

2

FP32 (Single Precision)

32-bit floating point format ensures:

- Stable accumulation of gradients
- Precise weight updates
- Numerical stability for sensitive operations

Mixed precision combines both to achieve the optimal balance between speed and accuracy

Key Benefits

3x

Faster Training

Up to 2-3x speedup on compatible hardware

50%

Memory Savings

Fit larger batch sizes in the same VRAM

95%

Hardware Utilization

Leverage specialized Tensor Cores on modern GPUs

100%

Accuracy Retention

Maintain model performance with careful loss scaling

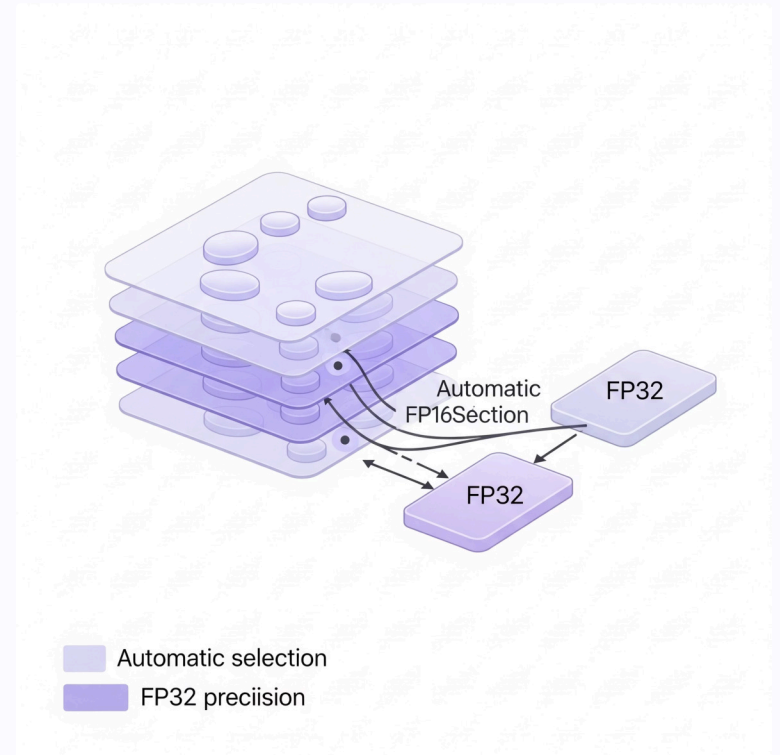


Automatic Mixed Precision (AMP)

Modern frameworks provide built-in AMP support:

- **PyTorch AMP** and **TensorFlow AMP** handle the heavy lifting
- Automatically casts operations to FP16 or FP32 where appropriate
- Keeps numerically sensitive operations (e.g., softmax) in FP32
- Dramatically reduces manual implementation effort

AMP makes mixed precision accessible to all practitioners without deep expertise in numerical computing.



Example – PyTorch AMP

```
scaler = torch.cuda.amp.GradScaler()
for data, target in dataloader:
    optimizer.zero_grad()
    with torch.cuda.amp.autocast():
        output = model(data)
        loss = criterion(output, target)
    scaler.scale(loss).backward()
    scaler.step(optimizer)
    scaler.update()
```



autocast()

Automatically chooses between FP16 and FP32 for each operation based on safety and performance



GradScaler()

Prevents numerical underflow in gradients by applying and adjusting scaling factors

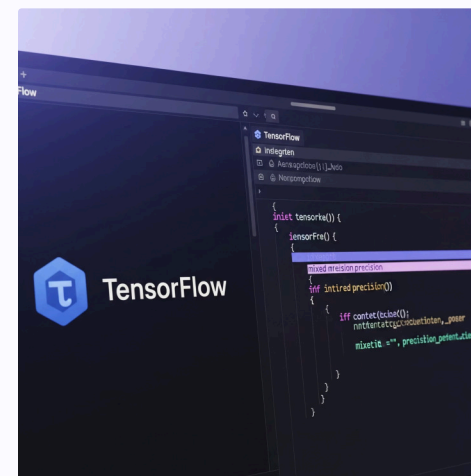
Example – TensorFlow AMP

```
policy = tf.keras.mixed_precision.Policy('mixed_float16')
tf.keras.mixed_precision.set_global_policy(policy)

model = tf.keras.Sequential([...])
model.compile(optimizer='adam', loss='categorical_crossentropy')
model.fit(train_ds, epochs=5)
```

Even simpler implementation:

- Just set the global policy to "mixed_float16"
- Framework automatically handles all casting
- Loss scaling is applied behind the scenes



Loss Scaling

Problem

FP16 has limited precision range ($\pm 65,504$), risking gradient underflow during backpropagation

Unscaling

Divide gradients by the same factor before optimizer update



Solution

Multiply loss by a large scaling factor (e.g., 128 or 256) before backprop

Scaling

Gradients become larger, preventing them from becoming too small for FP16

AMP frameworks handle this crucial process **automatically**, ensuring numerical stability



Tradeoffs & Considerations

Hardware Requirements

Works best on GPUs with **Tensor Cores** (Volta, Turing, Ampere architectures and newer)

Operation Support

Some operations aren't supported in FP16 and require automatic fallback to FP32

Debugging Challenges

Need to monitor for NaNs/Infs in training, which can indicate numerical issues

Validation Step

Always verify that accuracy matches FP32 baseline (usually does, but exceptions exist)

Real-World Use Cases

Computer Vision

ResNet-50 training on ImageNet achieves up to 3x speedup with mixed precision

NLP Models

GPT and BERT models rely heavily on mixed precision for feasible training times

Recommendation Systems

Enables larger batch sizes for improved convergence in collaborative filtering

Mixed precision has become essential for any deep learning at scale

Summary

Mixed precision training combines:

- FP16 operations for speed and memory efficiency
- FP32 operations for numerical stability

Automatic Mixed Precision (AMP) frameworks:

- Automate the tedious casting and scaling process
- Save valuable development time

The result: **Significant savings in time, memory, and compute costs** with no accuracy loss

Mixed precision has become standard practice in modern deep learning infrastructure

